Implementing an Elementary RoboCup

using **Symbolic AI** and **Logic Programming** with **Prolog**

This document provides a concise outline of your *group course project* along with helpful guidance. You are encouraged to form groups of four or five, delegating tasks and responsibilities as mutually agreed upon. However, it is essential that *all members* maintain a comprehensive understanding of the project, as each of you will be assessed on both the final deliverable and the project report.

This project serves as an evaluation of your proficiency in Symbolic AI and Logic Programming with Prolog, challenging you to design a simple RoboCup-inspired simulation.

Drawing upon the practical exercises explored throughout the course — including GPS, Scheduling, Interference, Tic-Tac-Toe, 8–Puzzle, STRIPS, Sudoku, FSM, Constraint Satisfaction Problems, etc, this assignment reinforces key concepts such as symbolic knowledge representation, logical reasoning, rule-based decision-making, search and planning, constraint satisfaction, and uncertainty.

You will design and implement a Prolog program to simulate and control players in a dynamic soccer environment, developing a basic system that integrates sensing, reasoning, planning, and action within a small simulated world.

This engaging hands-on experience aligns with RoboCup's educational objectives as a *benchmark challenge in AI research* providing an opportunity to apply theoretical knowledge in a practical and problem-solving context.

Begin by reviewing `Michael_Floyd_RoboCup_2008.pdf` — a concise presentation prepared for a 2008 thesis defense. Then, proceed to `Michael_Floyd_RoboCup_2012.pdf` which offers a more detailed overview of the challenge. The two documents outline a primarily Sub-Symbolic approach to simulation. However, I would like you to leverage your understanding of Symbolic AI to develop a small simulation using Prolog.

Having explored how Prolog is used for implementing engaging games, puzzles, and challenges, you can now apply these principles to your own simulation.

By the way, if you find yourself frustrated while navigating the two pdf documents meant to provide only an overview of the challenge — wondering why they include seemingly unrelated details that do not directly aid your project — know that this is intentional. Developing the ability to read efficiently and selectively is essential. In today's information–rich world, mastering the skill of filtering content, identifying what is truly relevant, and extracting key insights from vast amounts of material is invaluable.

Learning Objectives:

- Understand the basic functionalities of Prolog for building intelligent Symbolic AI systems.
- Create a Prolog program to simulate a simplified RoboCup-like environment, focusing on knowledge representation, logical reasoning, decision-making, rule-based inference, search and planning, constraint satisfaction, and uncertainty in Symbolic AI.
- Analyze and optimize symbolic AI strategies for game-playing and problem-solving agents by evaluating their performance in dynamic rule-based environments.

Key Components:

1. Setup:

- Familiarize yourself with the RoboCup simulation framework as outlined in the two provided resources.
- Review examples of sensory input and output, such as messages for sensing and agent commands, e.g., `move`, `kick`, `catch`.

2. Define the game environment:

- Represent the soccer field, players, ball, goals and roles (e.g., `forward`, `defender`, `goalkeeper`) symbolically using Prolog facts and relations about the field, positions, and player states. For example:

```
field( size(100, 50) ).
ball( position(20, 25) ).
player( team1, defender, position(10, 30), stamina(100) ).
```

- Start with a static environment before introducing dynamic changes, such as moving objects.

3. Implement the agent behaviors:

- Evaluate whether knowledge-based or goal-based agents would be more appropriate for addressing this challenge.
- Develop Prolog facts and rules for different player roles:
    - Goalkeeper: Catch and reset the ball.
    - Forward: Chase the ball and attempt to score.
    - Defender: Intercept opponents and pass the ball.
- Develop further rules for decision-making based on symbolic inputs; e.g., proximity to the ball, teammate positions, etc.

4. Simulate the game dynamics:

- Create rules to model game events dynamically, such as player movements, ball interactions, and scoring.
- Incorporate decision-making based on sensory input. For example:
    - If the ball is within a certain distance, decide whether to kick.

5. Test and Evaluate:

- Simulate a simple game scenario using your Prolog implementation.
- Evaluate your agent's performance using metrics such as goal-scoring, successful passes, and defensive actions.

These are only suggestions along with simple guidelines to help you get started. Your approach is entirely up to you, offering considerable freedom and flexibility in implementation. You can earn additional points by incorporating a graphical user interface, such as a Prolog library or extension, to enhance the game's visualization. However, a simple text-based execution — akin to the minimalistic style of Tic-Tac-Toe or Sudoku from your practical exercises — is also acceptable. Just make sure to deduct stamina when players move to reflect energy consumption, decide on a realistic kick range based on field size, implement goal detection logic to update the score when the ball reaches a goal, randomize or alternate turn order to prevent bias, ensure goalkeepers move toward the ball when necessary, add a mechanism to reset player positions after a goal, introduce pauses between rounds for better game observation, consider ball possession when determining if a player can kick or catch it, etc... Then:

1. Save the code as `robocup.pl` and then load/consult the file.
2. Run a few rounds of the simulation by calling, for example, `?- run_simulation(3).`
3. Observe the player movements, ball positions, etc…

So, begin by defining the essential elements of the soccer simulation, including the field, ball, players, and agent behaviors such as movement, kicking, and goalkeeping. Once these fundamentals are in place, enhance the system by incorporating rules for passing, scoring, stamina depletion, goal registration, and more sophisticated strategies as suggested above.

Below, I am providing **a suggested program structure** with helpful comments.

Additionally, you will find a **sample execution** of the program I have developed afterward.

```
% Define the soccer field dimensions.
field( … ).

% Define the ball's initial position.
ball( … ).

% Define player positions and states.
% Team 1 players:
player( … ).
player( … ).
player( … ).
% Team 2 players:
player( … ).
player( … ).
player( … ).

% Move towards the ball.
move_towards_ball( … ) :-
    player( … ),
    ball( … ),
    XDiff is X2 - X1,
    YDiff is Y2 - Y1,
    sign(XDiff, DX),
    sign(YDiff, DY),
    NewX is X1 + DX,
    NewY is Y1 + DY,
    retract( … ),
    assertz( … ),
    format('~w ~w moves to (~w, ~w)~n', [Team, Role, NewX, NewY]).

% Sign function to normalize movement.
sign(X, 1) :- X > 0.
sign(X, -1) :- X < 0.
sign(X, 0) :- X =:= 0.

% Kick the ball towards the goal.
kick_ball( … ) :-
    player( … ),
    ball( … ),
    abs(X1 - X2) =< 10, abs(Y1 - Y2) =< 10, % Relaxed kicking range for action
    goal_position( … ),
    XDiff is GoalX - X2,
    YDiff is GoalY - Y2,
    sign(XDiff, DX),
    sign(YDiff, DY),
    NewBallX is X2 + DX,
    NewBallY is Y2 + DY,
    retract( … ),
    assertz( … ),
    format('~w ~w kicks the ball to (~w, ~w)~n', [Team, Role, NewBallX, NewBallY]).

% Goal position depending on the team.
goal_position( … ).
goal_position( … ).

% Goalkeeper catches the ball if close enough.
catch_ball( … ) :-
    player( … ),
    ball( … ),
    abs(X - BX) =< 2,
    abs(Y - BY) =< 2,
    retract( … ),
    assertz( … ),
    format('~w ~w catches the ball at (~w, ~w)!~n', [Team, Role, X, Y]).
```

```
% Simulate one round of the game.
simulate_round :-
    % Attempt kicking first, prioritizing Team 1 then Team 2
    (kick_ball(team1, … ) ;
     kick_ball(team2, … ) ; true),
    % Move all players toward the ball
    move_towards_ball( … ),
    move_towards_ball( … ),
    move_towards_ball( … ),
    move_towards_ball( … ),
    % Goalkeepers attempt to catch.
    (catch_ball( … ) ; true),
    (catch_ball( … ) ; true),
    % Print current ball position.
    ball(position( … )),
    format('Ball is now at (~w, ~w)~n', [BX, BY]).

% Run simulation for N rounds.
run_simulation( … ).
run_simulation( … ) :-
    …,
    simulate_round,
    …,
    run_simulation( … ).
```

```
?- consult(robocup).
true.


?- run_simulation(3).
team1 forward kicks the ball to (21, 25)
team1 forward moves to (11, 29)
team1 defender moves to (29, 21)
team2 forward moves to (89, 29)
team2 defender moves to (69, 21)
Ball is now at (21, 25)
team1 forward kicks the ball to (22, 25)
team1 forward moves to (12, 28)
team1 defender moves to (28, 22)
team2 forward moves to (88, 28)
team2 defender moves to (68, 22)
Ball is now at (22, 25)
team1 forward kicks the ball to (23, 25)
team1 forward moves to (13, 27)
team1 defender moves to (27, 23)
team2 forward moves to (87, 27)
team2 defender moves to (67, 23)
Ball is now at (23, 25)
true ;
team1 forward moves to (14, 26)
team1 defender moves to (26, 24)
team2 forward moves to (86, 26)
team2 defender moves to (66, 24)
Ball is now at (23, 25)
true ;
team1 forward moves to (15, 25)
team1 defender moves to (25, 25)
team2 forward moves to (85, 25)
team2 defender moves to (65, 25)
Ball is now at (23, 25)
```

```
team1 forward kicks the ball to (24, 25)
team1 forward moves to (16, 25)
team1 defender moves to (24, 25)
team2 forward moves to (84, 25)
team2 defender moves to (64, 25)
Ball is now at (24, 25)
true ;
team1 forward moves to (17, 25)
team1 defender moves to (24, 25)
team2 forward moves to (83, 25)
team2 defender moves to (63, 25)
Ball is now at (24, 25)
true ;
team1 forward moves to (18, 25)
team1 defender moves to (24, 25)
team2 forward moves to (82, 25)
team2 defender moves to (62, 25)
Ball is now at (24, 25)
team1 forward kicks the ball to (25, 25)
team1 forward moves to (19, 25)
team1 defender moves to (25, 25)
team2 forward moves to (81, 25)
team2 defender moves to (61, 25)
Ball is now at (25, 25)
team1 forward kicks the ball to (26, 25)
team1 forward moves to (20, 25)
team1 defender moves to (26, 25)
team2 forward moves to (80, 25)
team2 defender moves to (60, 25)
Ball is now at (26, 25)
true ;
team1 forward moves to (21, 25)
team1 defender moves to (26, 25)
team2 forward moves to (79, 25)
team2 defender moves to (59, 25)
Ball is now at (26, 25)
true ;
team1 forward moves to (22, 25)
team1 defender moves to (26, 25)
team2 forward moves to (78, 25)
team2 defender moves to (58, 25)
Ball is now at (26, 25)
team1 forward kicks the ball to (27, 25)
team1 forward moves to (23, 25)
team1 defender moves to (27, 25)
team2 forward moves to (77, 25)
team2 defender moves to (57, 25)
Ball is now at (27, 25)
true ;
team1 forward moves to (24, 25)
team1 defender moves to (27, 25)
team2 forward moves to (76, 25)
team2 defender moves to (56, 25)
Ball is now at (27, 25)
true ;
false.
```

Deliverables:

- Prolog Code:
  - Submit a well-documented Prolog program implementing your solution.
  - Include clear and concise comments to enhance readability and understanding.
- A report (submitted in both WORD and PDF formats) limited to max 5 pages, including:
  - A description of your approach, covering symbolic representations and strategies as well as the rationale behind your design and implementation decisions.
  - An evaluation of your program's performance, identifying key strengths and specific areas for improvement.

Submission:

- myCourseVille

Deadline:

- 12:00 midnight on Sunday 19 April

Note:

- Plagiarism or the use of AI tools to produce content without demonstrating genuine understanding is strictly prohibited. Detection software will be used to identify both plagiarism and AI-generated materials.
- To ensure the authenticity of all work, students may be required to undergo an individual oral examination or "viva" regarding their submission. You may be quizzed on your code logic, report content, or specific technical choices to verify that you performed the work yourself and possess a true understanding of the material, regardless of the learning resources or tools utilized during the process.
- All work must also adhere to the University's Academic Integrity policies. As outlined in my course syllabus, I maintain a zero-tolerance stance on academic dishonesty. Any confirmed violation will result in an automatic failing grade for my section of the course. This policy is in place to uphold rigorous academic standards and ensure equity among all students.